

Bootcamp Project 2

Customer Churn Prediction

A Complete Hands-On Guide for Freshers

- > Data Exploration & Cleaning
- > Machine Learning (Decision Tree & Random Forest)
 - > Model Evaluation & Comparison
- > SQL Analytics (15 Business Queries)

Technologies: Python | Pandas | Scikit-Learn | Matplotlib | Seaborn | SQLite

Table of Contents

1.	Problem Statement	3
2.	Business Objectives	3
3.	What is Customer Churn?	4
4.	Dataset Description	4
5.	Project Structure & File Overview	5
6.	Prerequisites & Setup	5
7.	Step 1 - Data Exploration (EDA)	6
8.	Step 2 - Data Cleaning	8
9.	Step 3 - Decision Tree Classifier	10
10.	Step 4 - Random Forest Classifier	12
11.	Step 5 - Model Evaluation & Comparison	14
12.	Step 6 - SQL Queries for Business Analysis	16
13.	Key Findings & Conclusion	19
14.	Glossary of Terms	20

1. Problem Statement

A telecom company is facing a serious issue - many of their customers are leaving (this is called 'churn'). Every time a customer leaves, the company loses money. It costs 5 to 10 times more to get a new customer than to keep an existing one.

The company has collected data about 64,374 customers including their age, gender, how long they have been with the company, how often they use the service, how many times they called support, their payment delays, subscription type, contract length, total money spent, and whether they eventually left (churned) or stayed.

YOUR TASK: Build a machine learning system that can look at a customer's data and predict whether that customer is likely to leave or stay. This will help the company take action BEFORE the customer leaves - like offering discounts, better service, etc.

What exactly are we doing?

- **Step 1:** Explore the data - understand what information we have
- **Step 2:** Clean the data - fix any problems, prepare it for ML
- **Step 3:** Build a Decision Tree model to predict churn
- **Step 4:** Build a Random Forest model to predict churn
- **Step 5:** Compare both models - which one is better?
- **Step 6:** Run SQL queries to answer business questions

2. Business Objectives

The company wants answers to these key business questions:

- **Predict Churn:** Can we predict which customers will leave so we can take action early?
- **Identify Risk Factors:** What makes a customer leave? Is it bad support? High prices? Short contracts?
- **Customer Segmentation:** Which group of customers are at the highest risk of leaving?
- **Revenue Protection:** How much money are we losing due to churn? Who are our most valuable at-risk customers?
- **Improve Retention:** What changes can we make to reduce churn rate (better contracts, better support, etc.)?

Success Criteria

The project will be considered successful if:

- The ML model achieves at least 85% accuracy in predicting churn
- We can identify the top 5 factors that cause churn
- We can provide actionable business insights through SQL analysis
- All code is clean, well-structured, and easy for freshers to understand

3. What is Customer Churn? (Simple Explanation)

Imagine you have a Netflix subscription. If you cancel it, Netflix would say you have 'churned'. Customer churn simply means a customer has stopped using a company's service.

Real-World Example

Think of a mobile phone company like Jio or Airtel. They have millions of customers. Every month, some customers switch to another company. This is churn. If Airtel loses 10,000 customers per month and each customer pays Rs 500/month, they are losing Rs 50,00,000 (50 Lakhs) per month!

Why does Machine Learning help?

Instead of waiting for a customer to leave and then reacting, ML can predict WHO is likely to leave BEFORE they actually leave. This gives the company time to offer that customer a discount, better plan, or personal attention to make them stay.

Think of it like this:

A doctor does not wait for you to get very sick. They do regular checkups and predict problems early. ML does the same thing for businesses - it is a regular health checkup for customer relationships.

4. Dataset Description

Our dataset has 64,374 rows (customers) and 12 columns (features). Here is what each column means:

Column Name	Type	What it means (Simple Explanation)
CustomerID	Number	Unique ID for each customer (like a roll number)
Age	Number	Customer's age in years (18 to 65)
Gender	Text	Male or Female
Tenure	Number	How many months the customer has been with the company
Usage Frequency	Number	How often the customer uses the service per month
Support Calls	Number	How many times the customer called for help
Payment Delay	Number	How many days late the customer pays their bill
Subscription Type	Text	Basic, Standard, or Premium plan
Contract Length	Text	Monthly, Quarterly (3 months), or Annual (1 year)
Total Spend	Number	Total money the customer has spent (in dollars)
Last Interaction	Number	Days since the customer last contacted the company
Churn	Number	TARGET - 1 = customer left, 0 = customer stayed

NOTE: The 'Churn' column is our TARGET variable - this is what we want to predict.

Quick Dataset Statistics

- Total customers: 64,374
- Customers who stayed (Churn=0): 33,881 (52.6%)
- Customers who left (Churn=1): 30,493 (47.4%)
- The dataset is fairly balanced - almost equal number of churned and non-churned

5. Project Structure & File Overview

The project is split into separate files so each step is independent and easy to understand. Here is every file and what it does:

- **config.py** - Central settings file - all paths and constants in one place
- **01_data_exploration.py** - Step 1: Look at the data, understand patterns
- **02_data_cleaning.py** - Step 2: Fix problems in data, prepare for ML
- **03_decision_tree.py** - Step 3: Train Decision Tree model
- **04_random_forest.py** - Step 4: Train Random Forest model
- **05_model_evaluation.py** - Step 5: Compare models side by side
- **06_sql_queries.py** - Step 6: Run 15 business SQL queries
- **requirements.txt** - List of Python libraries needed
- **outputs/** - Folder where all plots, models, CSVs are saved

Why separate files?

In real projects, code is always split into modules. Each file has one job. This makes it easy to read, debug, and test. If there is a bug in data cleaning, you only look at 02_data_cleaning.py - you do not need to search through 1000 lines.

6. Prerequisites & Setup

What you need installed

- Python 3.8 or higher (download from python.org)
- pip (comes with Python automatically)
- A code editor - VS Code is recommended (free)

Step-by-step setup

Step A: Open terminal / command prompt in the project folder

```
cd Bootcamp-project-2
```

Step B: Install all required libraries

```
pip install -r requirements.txt
```

This installs: pandas, numpy, matplotlib, seaborn, scikit-learn

Step C: Run each file in order

```
python 01_data_exploration.py  
python 02_data_cleaning.py  
python 03_decision_tree.py  
python 04_random_forest.py  
python 05_model_evaluation.py  
python 06_sql_queries.py
```

NOTE: Run the files IN ORDER. Step 02 creates data_cleaned.csv which Steps 03-05 need.

7. Step 1 - Data Exploration (EDA)

What is EDA?

EDA stands for Exploratory Data Analysis. Before building any ML model, you MUST first understand your data. It is like reading the question paper before writing answers. In EDA, we look at:

- How big is the data? (rows and columns)
- What type of data do we have? (numbers, text, dates?)
- Are there any missing values?
- What does the data look like? (distribution, range, average)
- How is our target variable distributed? (balanced or imbalanced?)

What this code does (line by line)

- 1) Loads the CSV file into a pandas DataFrame (think of it as an Excel spreadsheet in Python)
- 2) Prints the shape (number of rows and columns)
- 3) Shows data types of each column
- 4) Shows first 5 rows to get a feel of the data
- 5) Calculates statistics (mean, min, max, etc.)
- 6) Checks for missing values
- 7) Shows how many customers churned vs stayed
- 8) Creates histograms for all numeric columns
- 9) Creates bar charts for categorical columns
- 10) Creates a correlation matrix heatmap

File: config.py

This file stores all paths and settings in one place so we do not repeat them.

```
"""Central configuration for paths and constants."""
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_RAW = os.path.join(
    BASE_DIR,
    "customer_churn_dataset-testing-master.csv",
    "customer_churn_dataset-testing-master.csv",
)
DATA_CLEANED = os.path.join(BASE_DIR, "data_cleaned.csv")
OUTPUT_DIR = os.path.join(BASE_DIR, "outputs")
os.makedirs(OUTPUT_DIR, exist_ok=True)

TARGET = "Churn"
RANDOM_STATE = 42
TEST_SIZE = 0.2
```

File: 01_data_exploration.py

Copy-paste this entire code into a file named 01_data_exploration.py:

```
"""
Step 1 - Exploratory Data Analysis (EDA)
Loads raw CSV -> prints shape, dtypes, stats, missing values, class balance.
Saves distribution plots to outputs/.
"""

import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from config import DATA_RAW, OUTPUT_DIR, TARGET

# Load the data
df = pd.read_csv(DATA_RAW)

print("=" * 60)
print("SHAPE:", df.shape)
print("=" * 60)

print("\nCOLUMN TYPES:")
print(df.dtypes)

print("\nFIRST 5 ROWS:")
print(df.head())

print("\nDESCRIPTIVE STATISTICS:")
print(df.describe())

print("\nMISSING VALUES:")
print(df.isnull().sum())

print(f"\nTARGET DISTRIBUTION ({TARGET}):")
print(df[TARGET].value_counts())
print(df[TARGET].value_counts(normalize=True).round(4))
```

01_data_exploration.py (continued) - Plotting Code

```
# Get all numeric columns (except CustomerID)
numeric_cols = df.select_dtypes(include="number").columns.tolist()
numeric_cols = [c for c in numeric_cols if c != "CustomerID"]

# Plot histograms for all numeric columns
fig, axes = plt.subplots(3, 3, figsize=(15, 12))
for ax, col in zip(axes.flatten(), numeric_cols):
    sns.histplot(df[col], kde=True, ax=ax)
    ax.set_title(col)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/numeric_distributions.png", dpi=150)

# Plot bar charts for categorical columns
cat_cols = df.select_dtypes(include="object").columns.tolist()
fig, axes = plt.subplots(1, len(cat_cols), figsize=(5 * len(cat_cols), 4))
if len(cat_cols) == 1:
    axes = [axes]
for ax, col in zip(axes, cat_cols):
    df[col].value_counts().plot.bar(ax=ax)
    ax.set_title(col)
    ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/categorical_distributions.png", dpi=150)

# Correlation heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(df[numeric_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Matrix")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/correlation_matrix.png", dpi=150)

print("EDA complete.")
```

What to look for in EDA output:

- 1) Shape tells you how much data you have. More data = better models.
- 2) Missing values must be fixed before ML. Our dataset has none - lucky!
- 3) The target is 52.6% vs 47.4% - this is balanced, which is good.
- 4) Correlation heatmap shows which features are related to each other.

8. Step 2 - Data Cleaning

Why do we clean data?

Real-world data is messy. It can have missing values, duplicate rows, extreme outliers, and text that machines cannot understand. Machine Learning models only work with numbers, so we need to convert text columns to numbers. Think of it like cleaning and organizing your desk before starting to study - you need a clean workspace.

What this code does

- **1. Remove duplicates:** Same row appearing twice adds no value
- **2. Fill missing values:** Numbers -> fill with median, Text -> fill with most common value
- **3. Handle outliers:** Extreme values are capped using the 1st and 99th percentile
- **4. Encode text to numbers:** Gender -> 0/1, Subscription Type -> one-hot encoding
- **5. Drop CustomerID:** It is just an ID, not a useful feature for prediction

What is One-Hot Encoding? (Simple Explanation)

Imagine Subscription Type has 3 values: Basic, Standard, Premium. We cannot feed text to ML models. One-hot encoding creates separate columns:

- Subscription_Type_Premium = 1 if Premium, else 0
- Subscription_Type_Standard = 1 if Standard, else 0
- If both are 0, it means Basic (this is why we use drop_first=True)

File: 02_data_cleaning.py

Copy-paste this code:

```

"""
Step 2 - Data Cleaning
Handles missing values, duplicates, outliers, type fixes.
Saves cleaned CSV to data_cleaned.csv.
"""

import pandas as pd
import numpy as np
from config import DATA_RAW, DATA_CLEANED, TARGET

df = pd.read_csv(DATA_RAW)
print("Raw shape:", df.shape)

# 1. Drop duplicates
before = len(df)
df.drop_duplicates(inplace=True)
print(f"Duplicates removed: {before - len(df)}")

# 2. Handle missing values
missing = df.isnull().sum()
print("\nMissing values per column:")
print(missing[missing > 0] if missing.any() else "None")

numeric_cols = df.select_dtypes(include="number").columns.tolist()
cat_cols = df.select_dtypes(include="object").columns.tolist()

for col in numeric_cols:
    if df[col].isnull().any():
        df[col].fillna(df[col].median(), inplace=True)

for col in cat_cols:
    if df[col].isnull().any():
        df[col].fillna(df[col].mode()[0], inplace=True)

```

02_data_cleaning.py (continued)

```
# 3. Outlier capping (IQR method)
cols_to_cap = ["Age", "Tenure", "Usage Frequency", "Support Calls",
               "Payment Delay", "Total Spend", "Last Interaction"]

for col in cols_to_cap:
    q1 = df[col].quantile(0.01)
    q99 = df[col].quantile(0.99)
    before_clip = ((df[col] < q1) | (df[col] > q99)).sum()
    df[col] = df[col].clip(q1, q99)
    if before_clip > 0:
        print(f"Clipped {before_clip} outliers in '{col}'")

# 4. Encode categorical features
df["Gender"] = df["Gender"].map({"Male": 0, "Female": 1})

df = pd.get_dummies(df, columns=["Subscription Type", "Contract Length"],
                    drop_first=True, dtype=int)

# 5. Drop CustomerID (not a feature)
df.drop(columns=["CustomerID"], inplace=True)

# 6. Save
df.to_csv(DATA_CLEANED, index=False)
print(f"\nCleaned shape: {df.shape}")
print(f"Saved to: {DATA_CLEANED}")
```

What is an Outlier?

An outlier is an extreme value. For example, if most customers spend \$200-\$800 but one spends \$99,999, that is an outlier. We clip (cap) extreme values to prevent them from confusing our ML model.

9. Step 3 - Decision Tree Classifier

What is a Decision Tree? (Simple Explanation)

A Decision Tree is like a flowchart of Yes/No questions. Imagine you are trying to decide what to eat for lunch:

- > Are you hungry? -> Yes
 - > Do you want something quick? -> Yes -> Order pizza
 - > Do you want something quick? -> No -> Cook biryani
- > Are you hungry? -> No -> Just have chai

A Decision Tree model works the same way! It asks questions about the data:

- > Is Payment Delay > 20 days? -> Yes
 - > Are Support Calls > 5? -> Yes -> CHURN (customer will leave)
 - > Are Support Calls > 5? -> No -> Maybe stay
- > Is Payment Delay > 20 days? -> No -> STAY (customer will stay)

Key Parameters Explained

- **max_depth=8:** The tree can go maximum 8 levels deep (prevents overfitting)
- **min_samples_split=20:** Need at least 20 samples to split a node further
- **random_state=42:** Ensures you get same results every time you run

What is Train-Test Split?

We split data into two parts:

- Training set (80%): Model LEARNS from this data
- Testing set (20%): We TEST the model on data it has NEVER seen

This tells us how well the model will work on new, unseen customers.

File: 03_decision_tree.py

Copy-paste this code:

```

"""
Step 3 - Decision Tree Classifier
Trains a Decision Tree on the cleaned data, saves the model.
"""

import pandas as pd
import pickle
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score,
)
from config import DATA_CLEANED, OUTPUT_DIR, TARGET, RANDOM_STATE, TEST_SIZE

# Load cleaned data
df = pd.read_csv(DATA_CLEANED)
X = df.drop(columns=[TARGET])    # Features (input)
y = df[TARGET]                  # Target (output)

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)
print(f"Train: {X_train.shape}, Test: {X_test.shape}")

# Create and train the Decision Tree
dt = DecisionTreeClassifier(
    max_depth=8, min_samples_split=20, random_state=RANDOM_STATE
)
dt.fit(X_train, y_train)    # This is where the model LEARNS

```

03_decision_tree.py (continued) - Evaluation

```
# Make predictions on test data
y_pred = dt.predict(X_test)
y_prob = dt.predict_proba(X_test)[: , 1] # probability of churn

# Print evaluation metrics
print("\n===== DECISION TREE RESULTS =====")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred):.4f}")
print(f"Recall : {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred):.4f}")
print(f"ROC-AUC : {roc_auc_score(y_test, y_prob):.4f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
                           target_names=["Not Churned", "Churned"]))

# Feature importance chart
feat_imp = pd.Series(dt.feature_importances_,
                    index=X.columns).sort_values(ascending=True)
plt.figure(figsize=(10, 6))
feat_imp.plot.barh()
plt.title("Decision Tree - Feature Importance")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/dt_feature_importance.png", dpi=150)

# Visualize the tree (top 4 levels)
plt.figure(figsize=(24, 10))
plot_tree(dt, max_depth=4, feature_names=X.columns,
          class_names=["No", "Yes"], filled=True, rounded=True, fontsize=8)
plt.title("Decision Tree (top 4 levels)")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/dt_tree_plot.png", dpi=150)

# Save model to disk
with open(f"{OUTPUT_DIR}/decision_tree_model.pkl", "wb") as f:
    pickle.dump(dt, f)
```

Expected Output

Accuracy: 0.9946 (99.46%)

Precision: 0.9936

Recall: 0.9951

F1 Score: 0.9943

ROC-AUC: 0.9999

10. Step 4 - Random Forest Classifier

What is a Random Forest? (Simple Explanation)

Remember the Decision Tree? A Random Forest is like asking 200 different Decision Trees and then taking a VOTE. Each tree is slightly different because:

- 1) Each tree sees a random subset of the data (like asking different people)
- 2) Each tree considers random features at each split

If 150 out of 200 trees say 'CHURN' and 50 say 'STAY', the final answer is 'CHURN'.

Think of it like this: Instead of asking one doctor, you ask 200 doctors and go with the majority opinion. This is almost always more accurate than one doctor alone.

Key Parameters Explained

- **n_estimators=200**: Number of trees in the forest (200 trees vote together)
- **max_depth=12**: Each tree can go 12 levels deep
- **min_samples_split=10**: Need at least 10 samples to split a node
- **n_jobs=-1**: Use all CPU cores for faster training

File: 04_random_forest.py

Copy-paste this code:

```
"""
Step 4 - Random Forest Classifier
Trains a Random Forest on the cleaned data, saves the model.
"""

import pandas as pd
import pickle
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score,
)
from config import DATA_CLEANED, OUTPUT_DIR, TARGET, RANDOM_STATE, TEST_SIZE

# Load cleaned data
df = pd.read_csv(DATA_CLEANED)
X = df.drop(columns=[TARGET])
y = df[TARGET]

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)
print(f"Train: {X_train.shape}, Test: {X_test.shape}")
```

04_random_forest.py (continued)

```
# Create and train the Random Forest
rf = RandomForestClassifier(
    n_estimators=200, max_depth=12, min_samples_split=10,
    random_state=RANDOM_STATE, n_jobs=-1
)
rf.fit(X_train, y_train)

# Make predictions
y_pred = rf.predict(X_test)
y_prob = rf.predict_proba(X_test)[:, 1]

# Print evaluation metrics
print("\n===== RANDOM FOREST RESULTS =====")
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"Precision : {precision_score(y_test, y_pred):.4f}")
print(f"Recall : {recall_score(y_test, y_pred):.4f}")
print(f"F1 Score : {f1_score(y_test, y_pred):.4f}")
print(f"ROC-AUC : {roc_auc_score(y_test, y_prob):.4f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
    target_names=["Not Churned", "Churned"]))

# Feature importance chart
feat_imp = pd.Series(rf.feature_importances_,
    index=X.columns).sort_values(ascending=True)
plt.figure(figsize=(10, 6))
feat_imp.plot.barh()
plt.title("Random Forest - Feature Importance")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/rf_feature_importance.png", dpi=150)

# Save model
with open(f"{OUTPUT_DIR}/random_forest_model.pkl", "wb") as f:
    pickle.dump(rf, f)
```

Expected Output

Accuracy: 0.9977 (99.77%)

Precision: 0.9990

Recall: 0.9961

F1 Score: 0.9975

ROC-AUC: 1.0000

Decision Tree vs Random Forest:

Random Forest is almost always better because it combines many trees. Decision Tree: 99.46% accuracy. Random Forest: 99.77% accuracy. The tradeoff is that Random Forest takes more time to train.

11. Step 5 - Model Evaluation & Comparison

Understanding Evaluation Metrics

Here is what each metric means in simple language:

- **Accuracy:** Out of all predictions, how many were correct? Example: If 100 customers were tested and 95 predictions were right, accuracy = 95%.
- **Precision:** When the model says 'CHURN', how often is it actually correct? High precision = fewer false alarms.
- **Recall:** Out of all customers who actually churned, how many did the model catch? High recall = we do not miss churners.
- **F1 Score:** The balance between Precision and Recall. If either is low, F1 will be low. Think of it as the overall grade.
- **ROC-AUC:** A score from 0 to 1 measuring how well the model separates churners from non-churners. 1.0 = perfect, 0.5 = random guessing.

What is a Confusion Matrix?

A confusion matrix shows 4 numbers:

- True Positives (TP): Predicted CHURN, actually CHURNED (correct!)
- True Negatives (TN): Predicted STAY, actually STAYED (correct!)
- False Positives (FP): Predicted CHURN, actually STAYED (false alarm)
- False Negatives (FN): Predicted STAY, actually CHURNED (we missed it!)

What is Cross-Validation?

Instead of one train-test split, we do 5 different splits (5-fold CV). The model is trained and tested 5 times, and we average the results. This gives a more reliable performance estimate.

File: 05_model_evaluation.py

Copy-paste this code:

```

"""
Step 5 - Model Comparison & Evaluation
Loads both models, compares them side-by-side.
"""

import pandas as pd
import pickle
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, precision_recall_curve,
    confusion_matrix, ConfusionMatrixDisplay,
)
from config import DATA_CLEANED, OUTPUT_DIR, TARGET, RANDOM_STATE, TEST_SIZE

# Load data and models
df = pd.read_csv(DATA_CLEANED)
X = df.drop(columns=[TARGET])
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
)

with open(f"{OUTPUT_DIR}/decision_tree_model.pkl", "rb") as f:
    dt = pickle.load(f)
with open(f"{OUTPUT_DIR}/random_forest_model.pkl", "rb") as f:
    rf = pickle.load(f)

models = {"Decision Tree": dt, "Random Forest": rf}

```

05_model_evaluation.py (continued) - Metrics & Plots

```
# Build comparison table
rows = []
for name, model in models.items():
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    cv = cross_val_score(model, X, y, cv=5, scoring="f1")
    rows.append({
        "Model": name,
        "Accuracy": round(accuracy_score(y_test, y_pred), 4),
        "Precision": round(precision_score(y_test, y_pred), 4),
        "Recall": round(recall_score(y_test, y_pred), 4),
        "F1 Score": round(f1_score(y_test, y_pred), 4),
        "ROC-AUC": round(roc_auc_score(y_test, y_prob), 4),
        "CV F1 (mean)": round(cv.mean(), 4),
        "CV F1 (std)": round(cv.std(), 4),
    })

comparison = pd.DataFrame(rows)
print(comparison.to_string(index=False))
comparison.to_csv(f"{OUTPUT_DIR}/model_comparison.csv", index=False)

# ROC Curve
plt.figure(figsize=(8, 6))
for name, model in models.items():
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    plt.plot(fpr, tpr, label=f"{name} (AUC={auc:.3f})")
plt.plot([0, 1], [0, 1], "k--", label="Random")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/roc_curve_comparison.png", dpi=150)

# Confusion Matrices side by side
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, (name, model) in zip(axes, models.items()):
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    ConfusionMatrixDisplay(cm,
        display_labels=["Not Churned", "Churned"]).plot(ax=ax)
    ax.set_title(name)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/confusion_matrices.png", dpi=150)
```

Model Comparison Results

Model	Accuracy	Precision	Recall	F1	AUC	CV F1 mean	CV F1 std
-------	----------	-----------	--------	----	-----	------------	-----------

Bootcamp Project 2 - Customer Churn Prediction Guide

Decision Tree	0.9946	0.9936	0.9951	0.9943	0.9999	0.9940	0.0006
Random Forest	0.9977	0.9990	0.9961	0.9975	1.0000	0.9979	0.0002

12. Step 6 - SQL Queries for Business Analysis

Why SQL?

SQL (Structured Query Language) is used to ask questions from data stored in databases. Even though we used Python for ML, business teams often use SQL to get quick answers. In this step, we load our CSV into an SQLite in-memory database and run 15 queries.

How it works

- 1) We load the CSV into a table called 'customers' in SQLite (no server needed)
 - 2) We run SQL queries on this table
 - 3) Results are printed in the terminal
- SQLite comes built-in with Python - no installation needed!

File: 06_sql_queries.py

Copy-paste this code:

```
"""
Step 6 - 15 SQL Queries on the churn dataset
Uses SQLite in-memory so no external DB server is needed.
"""

import sqlite3
import pandas as pd
from config import DATA_RAW

# Load CSV into SQLite
df = pd.read_csv(DATA_RAW)
conn = sqlite3.connect(":memory:")
df.to_sql("customers", conn, index=False, if_exists="replace")
```

All 15 SQL Queries with Explanations

Q01: Total number of customers

Counts all rows in the table.

```
SELECT COUNT(*) AS total_customers
FROM customers;
```

Result: 64,374 customers

Q02: Churn vs Non-Churn count

Groups by Churn (0 or 1) and counts each.

```
SELECT Churn, COUNT(*) AS count
FROM customers
GROUP BY Churn;
```

Result: Stayed=33,881 | Churned=30,493

Q03: Average age of churned vs non-churned

Calculates average, min, and max age for each group.

```
SELECT Churn,
       ROUND(AVG(Age), 2) AS avg_age,
       MIN(Age) AS min_age,
       MAX(Age) AS max_age
FROM customers
GROUP BY Churn;
```

Result: Non-churned avg=41.13 | Churned avg=42.90

Q04: Churn rate by Subscription Type

Which plan has the highest churn rate?

```
SELECT [Subscription Type],
       COUNT(*) AS total,
       SUM(Churn) AS churned,
       ROUND(AVG(Churn) * 100, 2) AS churn_rate_pct
FROM customers
GROUP BY [Subscription Type]
ORDER BY churn_rate_pct DESC;
```

Result: Basic=48.28% | Standard=47.33% | Premium=46.50%

Q05: Churn rate by Contract Length

Which contract type has the highest churn?

```
SELECT [Contract Length],
       COUNT(*) AS total,
       SUM(Churn) AS churned,
       ROUND(AVG(Churn) * 100, 2) AS churn_rate_pct
FROM customers
GROUP BY [Contract Length]
ORDER BY churn_rate_pct DESC;
```

Result: Monthly=51.61% | Annual=46.22% | Quarterly=44.05%

Q06: Average Total Spend by Churn status

Do churned customers spend less?

```
SELECT Churn,
       ROUND(AVG([Total Spend]), 2) AS avg_spend,
       ROUND(SUM([Total Spend]), 2) AS total_spend
FROM customers
GROUP BY Churn;
```

Result: Non-churned avg=\$560.54 | Churned avg=\$519.34

Q07: Top 10 highest spending customers

Shows the biggest spenders.

```
SELECT CustomerID, Age, Gender,
       [Total Spend], Churn
FROM customers
ORDER BY [Total Spend] DESC
LIMIT 10;
```

Result: Top spenders all have Total Spend = \$1000

Q08: Average Support Calls by Churn

Do churned customers call support more?

```
SELECT Churn,
       ROUND(AVG([Support Calls]), 2)
       AS avg_support_calls
FROM customers
GROUP BY Churn;
```

Result: Non-churned=4.5 calls | Churned=6.4 calls

Q09: High vs Low support calls - churn rate

Customers with >5 support calls vs others.

```
SELECT CASE
       WHEN [Support Calls] > 5
       THEN 'High' ELSE 'Low'
END AS support_level,
COUNT(*) AS total,
SUM(Churn) AS churned,
ROUND(AVG(Churn)*100, 2) AS churn_rate_pct
FROM customers
GROUP BY support_level;
```

Result: High support=60.99% churn | Low=32.92%

Q10: Usage Frequency buckets and churn rate

Groups customers by how often they use the service.

```
SELECT CASE
    WHEN [Usage Frequency] <= 10 THEN '0-10'
    WHEN [Usage Frequency] <= 20 THEN '11-20'
    ELSE '21+'
END AS usage_bucket,
COUNT(*) AS total,
ROUND(AVG(Churn)*100, 2) AS churn_rate_pct
FROM customers
GROUP BY usage_bucket
ORDER BY usage_bucket;
```

Result: Low usage (0-10)=54.96% churn | High usage=43%

Q11: Average Tenure by Churn status

How long do churned vs non-churned customers stay?

```
SELECT Churn,
    ROUND(AVG(Tenure), 2) AS avg_tenure
FROM customers
GROUP BY Churn;
```

Result: Non-churned=28.83 months | Churned=35.52 months

Q12: Short vs Long tenure churn rate

Customers under 12 months vs 12+ months.

```
SELECT CASE
    WHEN Tenure < 12 THEN 'Short (<12m)'
    ELSE 'Long (12m+)'
END AS tenure_group,
COUNT(*) AS total,
SUM(Churn) AS churned,
ROUND(AVG(Churn)*100, 2) AS churn_rate_pct
FROM customers
GROUP BY tenure_group;
```

Result: Short=32.24% churn | Long=50.30% churn

Q13: Churn rate by Gender

Is there a gender-based difference?

```
SELECT Gender,
       COUNT(*) AS total,
       SUM(Churn) AS churned,
       ROUND(AVG(Churn)*100, 2) AS churn_rate_pct
FROM customers
GROUP BY Gender;
```

Result: Female=55.05% churn | Male=38.58% churn

Q14: Average Payment Delay by Churn

Do late payers churn more?

```
SELECT Churn,
       ROUND(AVG([Payment Delay]), 2)
       AS avg_payment_delay,
       MAX([Payment Delay]) AS max_delay
FROM customers
GROUP BY Churn;
```

Result: Non-churned avg=12.45 days | Churned=22.33 days

Q15: High-risk customer segment

Customers with high support calls + low tenure + high payment delay.

```
SELECT COUNT(*) AS high_risk_count,
       SUM(Churn) AS churned,
       ROUND(AVG(Churn)*100, 2) AS churn_rate_pct
FROM customers
WHERE [Support Calls] >= 7
      AND Tenure < 15
      AND [Payment Delay] >= 20;
```

Result: 2,097 high-risk customers | 56.22% churn rate

Code to run all queries (add to 06_sql_queries.py)

```
# After defining QUERIES dict, run them all:
for title, sql in QUERIES.items():
    print("=" * 65)
    print(title)
    print("-" * 65)
    result = pd.read_sql_query(sql, conn)
    print(result.to_string(index=False))
    print()

conn.close()
print("All 15 SQL queries executed.")
```

13. Key Findings & Conclusion

ML Model Results

- Both Decision Tree (99.46%) and Random Forest (99.77%) achieved excellent accuracy
- Random Forest is the better model - higher accuracy, precision, recall, and AUC
- The most important features for prediction: Payment Delay, Support Calls, Contract Length
- Cross-validation confirms results are stable (not just luck)

Business Insights from SQL Analysis

- **Monthly contracts are risky:** 51.6% churn rate vs 44% for quarterly - push customers toward longer contracts
- **Support calls matter:** Customers with >5 calls have 61% churn rate vs 33% - improve first-call resolution
- **Payment delays are a red flag:** Churned customers delay 22 days vs 12 days - set up early payment reminders
- **Gender gap:** Female customers churn at 55% vs males at 38.6% - investigate gender-specific issues
- **Low usage = high risk:** Customers using service 0-10 times have 55% churn - engage inactive users
- **High-risk segment identified:** 2,097 customers with high support + low tenure + late payments: 56% churn rate - prioritize retention efforts here

Recommendations for the Business

- Offer discounts for customers switching from Monthly to Annual contracts
- Create an early warning system using the Random Forest model
- Improve support quality - reduce the need for repeated calls
- Send payment reminders before due dates to reduce delays
- Run targeted campaigns for inactive customers (low usage frequency)
- Give special attention to the 2,097 high-risk customers identified in Q15

14. Glossary of Terms for Freshers

- **DataFrame:** A table of data in Python (like an Excel spreadsheet). Created using pandas library.
- **Feature:** An input column used for prediction (e.g., Age, Tenure, Support Calls).
- **Target Variable:** The column we want to predict (Churn - 0 or 1).
- **Training Data:** The portion of data (80%) used to teach the model.
- **Testing Data:** The portion of data (20%) used to evaluate the model on unseen data.
- **Overfitting:** When a model memorizes training data but fails on new data. Like memorizing answers instead of understanding the concept.
- **Classification:** Predicting a category (Churn or Not Churn). Different from regression which predicts a number.
- **Encoding:** Converting text data to numbers so ML models can process it.
- **One-Hot Encoding:** Creating separate 0/1 columns for each category (e.g., Basic, Standard, Premium).
- **Outlier:** An extreme or unusual data point that is very different from the rest.
- **Correlation:** How strongly two variables are related. +1 = strong positive, -1 = strong negative, 0 = no relation.
- **Confusion Matrix:** A 2x2 table showing correct predictions (TP, TN) and mistakes (FP, FN).
- **ROC-AUC:** Area Under the ROC Curve. Measures how well the model separates classes. 1.0 = perfect.
- **Cross-Validation:** Testing the model multiple times with different splits to get a reliable performance estimate.
- **pickle:** A Python library to save a trained model to a file so you can reuse it later without retraining.
- **SQLite:** A lightweight database that runs inside your Python program. No server installation needed.